

e requests were approved and by whom
see merge request authors
see the latest CI/CD pipeline result for each merge request

The Compliance Report can be accessed in the top-level group by going to Security & Compliance
↳ Compliance Report.

21. Implementation of Correct User Permissions and Roles

Implementation of Correct User Permissions and Roles will have below Positive effects over the entire DevOps lifecycle in GitLab.

Restricting Developers

- To take major decisions like changing Security Policies
- Delete Issues and MRs
- Self Approve MRS or manage MR Approval rules
- Adding/Manage teams members to the projects
- Configure Webhooks
- Edit Project settings
- Assign/remove project from compliance framework
- Manage Push rules
- Turn on/off protected branches.

Risks: Users will be leveraged to perform the above activities which can be against best practices of GitLab

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/good-estimation-techniques/_index.md

Estimation techniques and approaches is one of the key areas that sets traditional process frameworks, such as Waterfall or V-Model, apart from Agile.

What is often causing confusion are Agile terms such as Story Points, Estimation Poker, T-Shirt Sizing, Velocity · and how all those terms relate to better and more precise estimates.

This brief How-To guide will quickly summarize the key issues addressed in Agile that enable teams to better and more reliably estimate their development efforts.

Note: We only suggest applying Estimation techniques for Projects over 2 months in duration.

Estimating GitLab Professional Services Projects

We recommend the below t-shirt sizing estimation if the GitLab and Customer Project teams decides to move forward with applying weight to GitLab Issues:

- Sm - 1 (0-2 hrs)
- Med - 2 (2-5 hrs)
- Large -3 (5-10 hrs)

anything more than 10hrs should be broken into smaller tasks

Estimation Uncertainty

It is worth noting that the meaning of "estimate" can be

1. to roughly calculate or judge the value, number, quantity, or extent of something \[verb\], and
2. an approximate calculation or judgment of the value, number, quantity, or extent of something \[noun\]

Synonyms include: calculate roughly, approximate, rough guess · hence the slang term "guesstimate" · a mixture of guesswork and calculation.

Dwight D. Eisenhower famously said: "Plans are worthless, but planning is everything. There is a very great distinction because when you are planning for an emergency you must start with this one thing: the very definition of "emergency" is that it is unexpected, therefore it is not going to happen the way you are planning."^[1]

The point is that estimates are never precise, but in project management · especially software / IT project management · estimates regularly are baselined, cast in stone, tracked against, and then cause many political problems because it is perceived that the people that originally estimated the effort got it all wrong.

They did not get it wrong, they just did not know enough at the time.

Cone of Uncertainty

The Cone of Uncertainty is a project management term used to describe the level of uncertainty existing at different stages of a project.

Whenever you estimate an effort at the very beginning, say at the Concept phase of an idea, you are likely to be magnitudes off in terms of precision. This is the point in time our Engagement Managers providing scoping information! During the negotiation phase.

As you go through different project phases and you learn more about the actual work and challenges involved, the better (meaning the more closely approximated to the actual work effort) your estimates will become.

In short, we become more certain of our estimates as we learn more about what we are estimating.

Estimation variability decreases the closer we get to completing the project.

The only exception to this is if you are working on a process that is 100% understood · a production line is a good example, you know exactly how long the piece will move through different stages on the production line and you can estimate exactly how long it will take to finish one instance of your product.

Production lines usually have been optimized with lots of empirical time measurements, ensuring

that each production step runs smoothly. PS engagement are inherently different.

If you are doing anything that involves uncertainty, creativity, or something that has never been before, the Cone of Uncertainty rules. This applies to most software development and IT projects.

Even regular IT tasks such as server upgrades frequently take longer than expected · may that be that your system administrator runs out of disk space, experiences an unexpected power outage that corrupts the upgrade, or some other "emergency".

You can automate the process and make it more predictable, such as Amazon Web Services automated server requisitions in the cloud, but this usually requires process standardization which basically means it is not something that involves uncertainty, creativity, or has never been done before · to the contrary, it is something that has been done thousands of times, is known, and is well understood.

Long and Short Term Planning Horizons

Decreasing estimation variability therefore is dependent on the planning horizon.

If I plan what I am going to do today, I most likely will have a good understanding of what my day looks like, and therefore I can estimate with high probability (or low estimate variability) that I am going to make it to an appointment at 5:00 PM this afternoon.

On the other hand, if I am trying to plan for something 5 months out, the best I can do is put down a reminder · the chances of something unforeseen happening that might change my plans are high. I might get sick, be out of town for unexpected business travel, or need to tend to my parents.

For estimation of software projects, this translates into the traditional "Planning Onion" as follows:

This "Planning Onion" basically works from the outside in to provide ever increasing levels of detail; as such it is closely related to the concept presented in the Cone of Uncertainty.

To reflect the timing aspect and estimation detail, you can think of the "Planning Onion" like this:

Finally, in terms of Agile / Scrum estimation time horizons, you can think of Long Term vs. Short Term planning horizons as follows:

Vision and Roadmap planning are Long Term Horizon activities, whereas Sprint planning falls into the Short Term Horizon activities.

Planning and Estimation Differences between Waterfall and Agile

So, what are the actual planning and estimation differences between Waterfall and Agile?

From an estimation technique perspective, there is no reason why T-Shirt Sizing or Estimation Poker could not be used on either methodology.

The main problem with Waterfall is that it attempts to estimate the entire project upfront, thereby committing the project to an unrealistic time line early on, without clearly understanding the scope.

The following picture shows a traditional Waterfall process with three imaginary (but in my experience very common) "interruptions", which basically reset the project to the beginning.

1. Requirements Change · key requirements changed and force the team back to the Analysis drawing board
2. Customer Management Turnover · your customer / client management or key stakeholders changed and the "new team" wants to provide input (read "new direction") on the project which causes a reset
3. Technology Innovation · new technologies allow for better ways to deliver the project, which forces the team to re-evaluate existing analysis, design, and code and start over

On waterfall projects, this happens all the time! You reset without delivering value.

Using Agile on the other hand, you are able to deliver small product increments rapidly. Your planning horizons are shorter, and the Agile team basically commits to delivering a working increment at the end of every sprint.

Even if requirements change mid sprint, the product increment at the end of that sprint will be delivered. New requirements make it into the product backlog to be prioritized for later sprints.

Even if customer / client management or stakeholders change, in Agile / Scrum the project team can demonstrate working software at the end of every sprint, many times lessening the desire to "change direction" (because, after all, it works already!). Any changes requested are logged as additional requirements in the product backlog to be prioritized for later sprints.

Lastly, even technology innovation can be accommodated by including it into the product backlog to be prioritized for later sprints.

The important thing here is that in Agile, despite interruptions, unexpected developments, and "emergencies", the team keeps delivering working software in the form of product increments at the end of every sprint · delivering real tangible value.

Contrast that with the Waterfall methodology, where the only value created often is a large set of documents that the team struggles to keep updated.

Delivering working software also creates "facts on the ground" that are harder to ignore or debate about than if the team had merely produced a set of documents.

Common Estimation Approaches

What are some common estimation approaches and how do they work?

Absolute vs. Relative Estimation

One key thing to keep in mind is that Agile / Scrum teams use relative estimation. This is because it is relative to the team. What does that mean? Essentially it means that the measurement unit / granularity agreed upon for a task is specific to the Development Team estimating it. Team A could estimate it at 8 story points, Team B might estimate 5 story points. It is relative to the team and a relative estimate does not try to be 100% precise by using absolute measure.

The following picture tries to clarify the differences:

The absolute measure here is milliliters (ml), which is an International System of Units fluid measure. There is no interpretation as to what it means. It is absolute. Wine bottles are classified accordingly.

The relative measure might be the small, medium, large classification used in T-Shirt Sizing or the 3, 5, 8 story point Fibonacci number sequence used in Estimation Poker (more about Estimation Poker in the next section).

The following table compares and contrasts absolute vs. relative estimation techniques:

{width="544" height="380"}

Estimation Poker

{width="523" height="285"}

As mentioned before, Estimation Poker is a relative estimation technique that favors accuracy over precision. It commonly uses the Fibonacci number sequence (1, 2, 3, 5, 8, 13, 21, 34, 55, ...) [²].

The Implementation Team members doing the work have to do the estimates; this is critical. You cannot have people other than the actual Implementation Team estimate the work. This is critical in order to create accountability and make sure the Implementation Team is fully committed to the estimate.

Estimation Poker self-corrects for team idiosyncrasies · meaning two teams might estimate the same work and come up with different estimates because the specific team's composition and skill set might be different.

The following 8 steps describe the basic Estimation Poker process:

STEP 1

Identify an "anchor story" · the best understood story everybody can agree on and provides a size / effort reference

STEP 2

Product Owner explains story

STEP 3

Each Development Team member selects a card (secretly)

STEP 4

Together, all Development Team members show their cards

STEP 5

If different, the HIGH and LOW estimators briefly explain their choice and assumptions

STEP 6

The Product Owner provides additional information, as necessary

STEP 7

The Development Team iterates the process, up to 3 times, until consensus is reached

STEP 8

Repeat for all stories that need to be estimated, always keeping the anchor story in mind

A common challenge is that with a Implementation Team of 7 (+/-2), you will experience disagreements. Some people will argue that a user story is a 5, others will bet it is a 13. So, what to do?

Common Resolution Approaches

There are basically 4 ways to resolve disagreement during the estimation process:

1. You can ask the outliers to plead their case and try to convince the other Development Team members. Basically this is a negotiation back and forth until consensus is reached. This kind of open discussion works well with equally balanced team members but can be challenging with strong , Type A, personalities and unbalanced teams, for example, a team consisting of 3 very strong technical engineers and 4 entry level engineers
2. You can do a silent vote or a Fist of Five consensus model
3. Majority votes are popular because the process avoids conflict, but majority votes usually end up running the risk of creating committee decision where nobody feel responsible · you want accountability, so this is not a good approach

4. You can choose the highest estimate, but this creates the risk of estimate bloat

The best way to resolve disagreements is to have the discussion and all team members agree on the estimate.

Remember that estimation is team specific, and that the estimate is basically the Implementation Team's commitment to deliver based on this estimate · so consensus is important.

All for one, and one for all!

T-Shirt Sizing and Affinity Estimation

{width="521" height="326"}

T-Shirt Sizing (aka Affinity Estimation) is based upon the need to estimate things quickly without necessarily having all the details available that you usually might require. Here are some examples of how T-Shirt Sizing can be used:

1. For budgeting purposes · when a Vision and Roadmap are being discussed, T-Shirt Sizing can do the trick by having some experienced developers look at the high level asks and sizing it appropriately. The same applies for staffing plans (how many PSEs do we need to hire next year?)
2. When user stories are not available · Program Manager / Project Managers are the detail level domain experts that know what their engagement is supposed to do in detail. During Estimation Poker, the Program Manager explains, discusses, and clarifies each user story. But, the reality is that oftentimes fully fleshed out user stories might not be available yet, nor will the Program Manager sometimes know 6 months ahead of the potential sprint what necessarily has to be developed. Having said that, at a minimum you should have a placeholder subject line for each user story you wish to estimate, for example, "Support HP OfficeJet 9800" · at least developers will quickly be able to grasp what is required in the larger scheme of things without knowing the exact details
3. When you have 2,986 user stories · it is not uncommon for a productive Program Manager / Project Manager to crank out hundreds and even thousands of user stories · at varying levels of detail, some fully fleshed out, and some only with a subject line. Then the challenge becomes the excessive amount of time it would take to estimate all those stories. Do you put your development effort on hold for 6 weeks simply to generate estimates with Estimation Poker?

The answer to these estimation challenges is to use T-Shirt Sizing. In order to have a somewhat comparable scheme between Estimation Poker and T-Shirt Sizing, it is suggested to use the following quantification scheme:

XS = 1 story point

Small = 2 story points

Medium = 3 story points

Large = 5 story points

XL = 8 story points

XXL = 13 story points

EPIC = Program Manager / Project Manager needs to break down the Epic into more refined User Stories (for example, the EPIC might state something like "Internationalization Support". This needs to be broken down into more detailed user stories like "Provide Support for British

English", "Provide Support for Spanish", in order to be able to estimate it)

Then, follow these four easy steps:

STEP 1

Take 1 minute and have the Development Team agree on a single "small" user story. Repeat with XS, medium, large, XL, XXL, and EPIC sized stories

STEP 2

Take 10 · 30 minutes and have the Development Team put all remaining user stories into categories of XS, small, medium, XL, XXL, and EPIC

STEP 3

Take another 10 · 30 minutes and have the Development Team review and adjust user story categorizations as necessary · DONE!

STEP 4

Have the Product Owner review the categorizations

This 4 step process allows the Development Team to tackle many hundreds of user stories quickly.

Regardless if you are using Estimation Poker, or T-Shirt Sizing, the Cone of Uncertainty still applies. Be aware that using T-Shirt Sizing early in the process will produce estimates with higher variability.

Depending on what you are challenged with estimating, choose the right estimation method:

Roadmap Planning => Use T-Shirt Sizing

Release Planning => Use T-Shirt Sizing or Estimation Poker, depending on the number of user stories

Sprint Planning => Use Estimation Poker

Estimation Team Size

The estimation team size is determined by the Agile Implementation Team size, which should be 7 (+/-2).

The people that will do the work, need to be the ones that estimate it.

Sometimes organizations think that adding more people to the estimation effort will make the estimate better, but that is a commonly accepted fallacy. Larger teams do not produce better estimates:

{width="600" height="377"}

Estimation accuracy actually decreases with increasing team size.

Extensive research has been done on team size considerations and it is generally acknowledged that the optimal team size is somewhere between 5 to 8 team members. Some teams have been shown to function well with up to 12 team members. Co-location, offshoring, partnering, and staff qualification all affect the estimation efficiency.

It is worthwhile pointing out, that sports teams are organized around this size (Soccer: 11; Basketball: 5; Baseball: 9; Football: 11). The exception seems to be Rugby, which has 15. Military squads are organized around this size as well.

What is important to understand, both for estimation efficiency as well as team dynamics, is the following:

The bigger the team, the harder it is to effectively communicate, estimate, and come to consensus. The bigger the team, the more likely something will fall through the cracks because of miscommunication. Because most of our PS engagements are conducted remotely, this problem is amplified even more.

This challenge of ever increasing communication paths with larger teams is also somewhat related to the famous "Dunbar's number", which suggested a cognitive limit to the number of people with whom one can maintain stable social relationships (~150).

What is Velocity?

Velocity is the rate at which the Development Team can reliably deliver story points (usually packaged into time boxed sprints and resulting in a working product increment).

Estimates Self-Correct

Using Estimation Poker, estimation variability decreases usually over the course of the first 4 to 6 sprints. I have rarely seen stable teams that did not achieve fairly accurate estimates after working together for 5 to 6 sprints.

Please note that "stable teams" is key here · teams need to be stable, meaning team members need to know each other, have worked with each other, and successfully formed a team · following the standard "forming·storming·norming·performing" model of group development, first proposed by Bruce Tuckman in 1965.

These phases are all necessary and inevitable in order for the team to grow, to face up to challenges, to tackle problems, to find solutions, to plan work, and to deliver results. As most of our larger PS engagement will require extensive cross-team communication and collaboration with Customer Development Teams, it is necessary to understand and be aware of team dynamics on the customer side as well.

Estimate Accuracy and Velocity over Time

Once a Implementation Team has gone through the "forming·storming·norming·performing" process, team velocity is established. Estimation accuracy and predictable velocity allow for longer term forecasting.

Simply put, if your product backlog contains 450 user stories representing 2,400 story point and your Development Team is delivering at a steady velocity of 60 story points for every 2 week sprint, you are able to predict that the rest of the project will take another 40 sprints, which equals 80 weeks, or roughly 1 ½ years.

$$2,400 / 60 = 40 \text{ Sprints}$$

$$40 \text{ Sprints} \times 2 \text{ week sprint duration} = 80 \text{ weeks}$$

$$80 \text{ weeks} / 52 \text{ weeks in a year} = 1.54 \text{ years}$$

Accordingly, calculating your burn rate becomes a trivial exercise. Simply multiply your labor cost and overhead by the duration. For example, if you know that your 7 member Development Team costs you \$1,529,500 per year (\$805,000 for salaries, \$724,500 for overhead / fringe benefits, etc.) then you can easily figure out the cost per sprint ($\$1,529,500 / 26 = \$58,827$).

You cannot get any better at forecasting and predictability.

Summary

Agile / Scrum estimation using Estimation Poker is an empirical, self-correcting, approach to short term software estimation where user stories are fully defined.

Because it is an empirical approach to project planning, you always know exactly where your project stands at any given time.

T-Shirt Sizing (Affinity Estimation) is used for higher level estimates needed for budgeting, staffing, and other longer term planning activities where not all user stories are defined in all detail and a quick assessment of upcoming work is needed.

[^1]: From a speech to the National Defense Executive Reserve Conference in Washington, D.C. (November 14, 1957) ; in Public Papers of the Presidents of the United States, Dwight D. Eisenhower, 1957, National Archives and Records Service, Government Printing Office, p. 818 : ISBN 0160588510, 9780160588518

[^2]: By definition, the first two numbers in the Fibonacci sequence are 0 and 1, and each subsequent number is the sum of the previous two

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/good-user-stories/_index.md

Quick and Dirty Way to Write Good User Stories

User stories serve one main purpose: To encourage conversation and discussion about the user story. By design, user stories are supposed to be simple and short descriptions of features to be developed; written from the end user's perspective, in the end user's language. This is important to remember. User stories facilitate conversation and discussion in order to create a common understanding of the desired functionality amongst the team members. User stories are NOT comprehensive specifications.

User stories typically follow a simple template:

md

As a <user> I want to <action> so that <result / benefit>

Basic understanding of the user story requires all team members to be clear on:

- [] Who?... As a \<user\>
- [] What?... I want to \<action\>
- [] Why?... So that \<result / benefit\>

Every team member is supposed to be able to explain "Who?", "What?", and "Why?" for each user story. It's important to explicitly call this out because unless each team member understands the essence of each story, the team as a whole is lacking the necessary understanding of the problem space. Note that not each team member needs to understand how to implement the user story, but each member should understand the essence - for example, I do not need to know the details how to build a complex multi-stage CI yaml file, but I do need to know that our customer has a Dev, Test, Integration, Security, Staging, and Production stage setup that progressively stabilizes the system.

User stories should follow the INVEST principle. They should be:

Independent (to the degree possible, user stories should not rely on other user stories to be implemented)

Negotiable

Valuable

Feature focused, not task oriented

Written in the user's language to the extent possible; where domain specific language is required, such as in genomics or medicine, a subject matter expert will have to be available

Estimable

Small

Half a Sprint or less for one Dev Team member

Testable

Needs to have acceptance criteria and not be subjective

The testability of a user story is usually documented in the form of Acceptance Criteria and lists pass/fail testable conditions that help us verify if the user story is implemented as intended.

All sounds simple enough, but frequently teams and Product Owners really struggle with writing

good user stories, and 90% of all problems encountered are based on user stories being too big.

When a user story is too big, it is hard understand, estimate, implement, and prone to wildly different opinions. So what is a good size for a user story? Basic guidance is that the user stories at the top of the product backlog (the ones that should have sufficient specificity in order to be worked on by a PSE) should be sized such that a team member could easily complete two user stories during a sprint.

When a team's user stories are smaller, the team complete stories more frequently. One of the great side effects of smaller user stories is that the team gets more frequent feedback, more frequent successes, and the burndown charts are more granular and look more like a graph instead of a set of stairs.

How does a team take the big stories in its product backlog and split them into smaller stories? There are 3 basic techniques that we can use to split user stories.

1. Splitting stories with generic words
2. Splitting stories by acceptance criteria
3. Splitting stories with conjunctions and connectors

Splitting User Stories with Generic Words

This approach simply looks out for words that are open to a wide interpretation (i.e. that are too "generic") or have multiple meanings. For example, if you read a sentence like "We are moving all infrastructure to the cloud", what does that mean? You might have seen or heard sales pitches that sound great, but 2 minutes later you set back and think to yourself "I have no idea what that means!".

What is infrastructure? Servers? Desktops? Laptops? Only database servers? Buildings? Are you getting rid of all your data centers? Some? Move from desktops / laptops to Chromebooks?

What cloud? Amazon's AWS? Microsoft's Windows Azure? IBM's Bluemix? Public? Private? Hybrid?

What SSO? SAML, OIDC, OAuth, JWT, others?

The point is the generic words lack specificity. Perhaps an example would help explain this:

- > As a credit card transaction,
- >
- > We want activities to be logged,
- >
- > So that the account ledger is up to date.

In this story, the word "activities" is pretty generic. We can replace "activities" with more specific words such as: debits, credits, voided transactions. We will get these stories.

- > As a credit card transaction,
- >
- > We want debits to be logged,
- >

> So that the account ledger is up to date.
>
> AND
>
> As a credit card transaction,
>
> We want credits to be logged,
>
> So that the account ledger is up to date.
>
> AND
>
> As a credit card transaction,
>
> We want voided transactions to be logged,
>
> So that the account ledger is up to date.

By providing specificity, you enable to team members to realistically assess what is needed for an implementation, and, more importantly, you avoid ambiguity where individuals can interpret the same user story in different ways.

Splitting User Stories by Acceptance Criteria

This method looks at the acceptance criteria listed on the back of a user story card to split user stories into smaller, digestible chunks. What are acceptance criteria? Each user story should have between 6 and 12 acceptance criteria. The Product Owner works with the team to create, agree-upon, and record the acceptance criteria for each user story before the story enters a sprint. Think of this as the "definition of done" at the user story level.

Again, let's look at an example:

> As a credit card user,
>
> I want voided transactions to be logged,
>
> So that the account ledger is up to date.

Here are some acceptance criteria for this story:

Transaction support US dollars
Transactions support Canadian dollars
Transactions do not support Euros
Transactions support Visa Cards
Transactions support MasterCard
Transactions support American Express cards
Transaction do not support Diners Club cards
No transactions above \$500

If we examine each one of the acceptance criteria, we can ask "Who wants this?" The answer to

this question becomes the user in "As a (type of user)."

Next, we ask "Why do they want that?" The answer to this question identifies the value in "so that (some value is created)."

The body of the acceptance criteria provides the "I want (something)" part. Here are user stories that could be derived from the acceptance criteria above.

- > As a Visa credit card user,
- >
- > I want US dollar voided transactions to be logged,
- >
- > So that the account ledger is up to date.
- >
- > AND
- >
- > As a Visa credit card user,
- >
- > I want Canadian dollar voided transactions to be logged,
- >
- > So that the account ledger is up to date.
- >
- > AND
- >
- > As a MasterCard credit card user,
- >
- > I want US dollar voided transactions to be logged,
- >
- > So that the account ledger is up to date.
- >
- > AND
- >
- > As an American Express credit card user,
- >
- > I want Canadian dollar voided transactions to be logged,
- >
- > So that the account ledger is up to date.
- >
- > AND
- >
- > As a Diners Club credit card user,
- >
- > I want to receive an error message denying the transaction,
- >
- > So that the user has a clear understanding that Diners Club is not supported.
- >
- > ETC.

Using acceptance criteria to break up bigger user stories into more digestible ones is simple and quick and helps refine the backlog so that ambiguity is driven out. It also allows the

Product Owner to be more effective in prioritizing work; in our above example he/she might decide to implement Visa Card support in Sprint 1, MasterCard support in Sprint 2, but delay American Express card support to Sprint 7 · depending on the business needs and value each provides.

Splitting User Stories with Conjunctions and Connectors

This approach simply encourages the team members to review the user stories and look for connector words such as: and, or, if, when, but, then, as-well-as, etc. Commas and semicolons often function as connectors.

It is usually safe to assume that you can split the user story into two user stories by separating the pieces on each side of the connector words, for example:

- > As a credit card user,
- >
- > I want US dollar voided transactions to be logged, as well Canadian dollars, but not Euros,
- >
- > So that the account ledger is up to date.

You can easily make this into three distinct user stories:

- > As a credit card user,
- >
- > I want US dollar voided transactions to be logged,
- >
- > So that the account ledger is up to date.
- >
- > AND
- >
- > As a credit card user,
- >
- > I want Canadian dollar voided transactions to be logged,
- >
- > So that the account ledger is up to date.
- >
- > AND
- >
- > As a credit card user,
- >
- > I want to receive an error message denying the transaction when trying to void a transaction in Euros,
- >
- > So that the user has a clear understanding that Euros are not supported.
- >
- > Of course, each of these in turn can be analyzed based on other criteria (Visa Card user, MasterCard user, etc.).

That's pretty much it! Simple. Although there are other techniques to analyze user stories, the three mentioned above will get most teams to write much better, more precise, and smaller user

stories.

Because of the more granular specificity, the implementation team will have a better chance of actually implementing the right thing, while providing the Product Owner with the opportunity to prioritize the backlog in a more optimal way.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/iteration-0/_index.md

Quick Links: | Iteration 0 Fundamentals | Delivery Kits

Phase Overview

Iteration 0 consists of four key activities that build upon each other:

1. EM>PS Transition - Internal knowledge transfer
2. Stakeholder Planning Meeting - Initial customer alignment
3. Customer Kickoff - Full team engagement
4. Support Preparation - Proactive issue management

Each activity has specific inputs, processes, and outputs to ensure a smooth start to your project.

Stakeholder Planning Meeting

Purpose: Align on project scope, resources, management approach, and dependencies to uncover any inconsistencies in expectations before the full team kickoff.

Meeting Setup

1. Schedule a dedicated meeting with Customer PM & key stakeholders from both GitLab and Customer teams.
2. Use the Stakeholder Planning Template to guide the discussion
3. Focus on mutual understanding of project parameters

Key Discussion Topics

1. Project Stakeholders
 - Identify all key participants and their roles
 - Confirm availability and communication channels
2. Project Objectives
 - Validate business drivers and success criteria
 - Align on use cases and expected outcomes
3. Project Velocity and Timeline

- Set expectations on pace and milestone dates
- Discuss sprint/iteration cadence preferences

4. Project Prerequisites

- Review technical and organizational readiness
- Identify any blockers to starting work

5. Kickoff Preparation

- Determine agenda and participants for the full kickoff
- Set expectations for the kickoff meeting

6. Onboarding Validations

- Ensure all access and permissions are in place
- Verify environments and tooling availability

7. Next Steps

- Document action items with owners and deadlines
- Plan for Customer Kickoff and Discovery sessions

Key Outputs

- [] Validated stakeholder list and roles
- [] Aligned expectations on timeline and velocity
- [] Documented prerequisites and dependencies
- [] Prepared agenda for Customer Kickoff
- [] Action items with clear ownership
-] Create Customer Slack channel and invite Customer Project team members. You can you use this AR as [template

> · Tip: Use this meeting to identify and address any expectation misalignment between GitLab and the Customer team before the broader kickoff meeting.

Prepare Support for Issues

Purpose: Proactively prepare the Support team by providing key project information to streamline issue resolution during the engagement.

> This process is critical for infrastructure-related projects like implementations but valuable for any engagement where support issues may arise.

ZenDesk Access

If you don't have ZenDesk light (Read-Only) access:

- For Zendesk Global
 - See Requesting a ZenDesk ·Light Agent· account
- For Zendesk US Government
 1. Open an Access Request

1. Make sure the System name value is Zendesk - US Federal, light agent access
1. Assign to your manager
1. Once your manager approved (as per the AR instructions), they will ping the Zendesk - US Federal techstack owner(s)
 - Keep in mind you must have US citizenship to have this access

Creating a Support Note

```
{{% alert title="Note" color="danger" %}}
```

The below information is solely for Zendesk Global. If the organization is on Zendesk US Government, please make a post in supportoperations specifying you need to modify an organization note in Zendesk US Government, making sure to provide the Salesforce Account or Zendesk Organization link (do not state the name of the customer or any other sensitive information).

Customer Support Operations (those with Zendesk US Government access) will work with you to get that completed.

```
{{% /alert %}}
```

Step 1: Find the Customer Organization

1. Access the Organizations Repository
2. Search for the Customer Name (will appear as a hash followed by the Salesforce name)

Step 2: Edit the YAML File

1. Select the YAML from Search, then click Edit > Open in Web IDE

2. Add information after the notes section (starting with a pipe |):

```
yaml
---
id: 27946339528
name: 5a1f9965 Test Account
notes: |
  PS Project in Progress
  Project Manager:
  Slack Channels:
  Engineers:
  Start Date:
  Anticipated End Date:
  Summary of Engagement:
  Support should know:
  Collaboration Project RAID(Issue) Board Link:
```

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/iteration-0-fundamentals/_index.md

Team Mobilization

1. Align on team norms and working agreements.
2. Orient the team on roles and responsibilities.
3. Agree on working cadences.
4. Align on vision, scope, and goals for your specific team.

Working Agreements

As a PMO organization, we assemble micro teams on a consistent basis, with both internal GitLab and external GitLab partners.

When a delivery team comes together for a new project, a short ritual is held to agree to working norms; ultimately fostering a positive working relationship that translates to successful project delivery and enhanced interpersonal, working relationships.

This falls in line with several of our GitLab values and promotes Psychological Safety. The following link documents GitLab's view of Psychological Safety.

Why working agreements?

Working agreements install ownership and agency, contributing to:

- stronger team cohesion and collaboration.
- increased productivity.
- early identification of project issues through regular communication.
- better communication and transparency.
- better work-life balance.

Project team consists of Professional Services Engineers, Technical Architects, and Program/Project Managers.

Making the implicit explicit promotes a connected experience among team members and builds psychological safety.

Working agreements are NOT be a tool to gauge performance. They are ONLY intended to help the team work together.

Example Working Agreements

We assume positive intent and acknowledge that throughout the entire project, each team member will do the best they can, given their experience level and perspective.

We agree to make our work transparent to our colleagues and customers using GitLab

Collaboration Project and collaboration tools.

Mark yourself as "away" when not available, with expected return time.

We will update issue status and time tracking daily.

Core collaboration hours: 10:00 AM - 4:00 PM EST; no expectation of response outside working hours.

We acknowledge GitLab's preference for Asynchronous Communication. This is NOT a directive to define working hours, only an FYI to your teammates.

We honor our commitments to our teammates and to our customers. If we commit to doing something, we follow through within expected timelines.

We understand that working agreements are not authoritative, but fluid and open to change when necessary.

Engagement Planning

Starts at the EM>PS Delivery Transition and continues through the Stakeholder Planning Meeting & Customer Kickoff. Throughout Iteration 0, work to continuously...

1. Refine and prioritize project backlog (user stories and acceptance criteria - two iterations).
2. Slot issues into an initial release plan (initial planning & design).
3. Define Definition of Done (DoD) and, if needed, the Definition of Ready (DoR).
4. Understand constraints, impediments and risks for your specific team.

Setup of Technical and Business Foundation

1. Identify key risks and dependencies based on initial backlog and tools.
2. Design data and business architectures and CI/CD pipeline to support desired functionality.
3. Define and implement engineering best practices to enable DevSecOps.

Prep for Production Readiness

1. Identify key risks and dependencies based on initial backlog and tools.
2. Design data and business architectures and CI/CD pipeline to support desired functionality.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/iteration-planning-per-service-offering/_index.md

Iteration 0 Planning for Transformational/Large scale Migrations

As we head into Kickoff and Planning & Design with our customers, its important that we're able to articulate what we plan to cover within the initial planning phases, and how it will impact the overall project and roll into future Iterations. The below guide includes the management principles across the Iterations to ensure operational excellence as we move into delivery. This allows us to show measurable results early and often to emphasize the importance of the

initiative and continually gain buy in from stakeholders. We need to teach the core Customer team how to use GitLab to manage large initiatives, to help reinforce the Customer's decision to leverage the GitLab platform.

In all Transformation projects, there will be elements of SCM migration and CI transition. Focus on creating a structure that is detailed and specific to drive adoption of those key GitLab features.

1. Iteration 0:

1. Source/destination discovery and access investigation
definition of done: tokens gathered from each system.
2. Congregate container/VM deployed
definition of done: congregate running. Tokens added. Validated API calls return data, validating network between source, destination, and congregate VM.
3. Plan waves: establish spreadsheet to plan waves, or inspect source system to outline specific root groups to migrate. Create a tracking list to understand what projects will be moved on which date. Understand users affected by these waves. Establish CI/CD conversion dependencies as part of wave plan. Establish migration validation criteria and execution
4. Establish communication with end users
Definition of done: customer program manager understands maintenance window (for source being in read-only mode), which projects will move on which date, which users will be affected by which projects, where users are supposed to go for support or if they have questions.
5. Plan CI/CD pilot application teams. Hold discovery sessions with each application team who have been identified as a pilot team.
6. PM Review
Retro on communication, collaboration, expectations management, etc.
Iteration review on items completed/blockers

2. Iteration 1 - N. In subsequent iterations prior to or in parallel with scaled migration, map the pilot activities and migration requisite issues into each iteration:

1. Hold Planning session with CI pilot team to understand job-level
definition of done, e.g. what are the inputs and outputs that tell us that the job was translated correctly; also non functional requirements like runtime/performance.
2. Hold working sessions to translate Jenkins/Teamcity/Bamboo etc. jobs into GitLab CI
definition of done: use above definition of done criteria as evidence of completion.
3. Document learnings into reusable instructions to be used in future scaled transition efforts.
4. Memorialize pilot team CI transition with key stakeholders and provide demonstration
5. Template CI pipeline jobs into CI template library. Refactor pilot team CI pipeline to point to templates.
6. Create issue template that migration automation will open upon SCM migration with step by step guidance on how to translate CI and/or use GitLab CI template library.
7. Hold widespread lunch and learn to showcase pilot team transition and highlight the use of CI template repository.
8. PM review:
Retro on application selection for Pilot, time dedicated from customer pilot team members, bureaucracy or other constraints that were encountered, etc
Iteration review - demo the progress on the pilot team CI translation, discuss next steps and blockers.

3. Iteration N - M. For iterations during migration:

1. Create one issue per wave for migration engineer to use as runbook. Communicate to end

users who will be affected by that wave. Communicate how they can get support and what to expect in terms of the new issue landing in their newly-migrated GitLab project.

2. Create the dashboard that can show the burnup of issues closed (indicating CI adoption). Create another dashboard (Grafana?) that shows CI jobs run per day. Showcase to key executives once any amount of data starts to show up.

3. Hold office hours to help customers drive adoption of GitLab CI.

4. Create a Consultation request approach for an app team to request a GitLab Engineer to help them translate their pipeline job?

5. PM:

1. Schedule stakeholder review / Steerco.

2. Retro - review support requests, issues in step by step communications, data integrity post migration, etc. Iteration review to showcase the number of apps migrated and number of CI jobs run/ CI issues closed.

4. Iteration M - O. Iterations after the SCM migration but during CI transition

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/iteration-scheduling/_index.md

Cadence Planning

Although the above pictures show a one-week and two-week sprint (aka iteration) it is assumed that iteration duration can be adjusted based on customer needs - as long as the duration stays consistent.

Generally established best practice is to not create iterations of varying durations as this prevents consistent measurement from one sprint to the next (velocity will appear skewed). Also, sprint durations between 1 week and 4 weeks are most common. This sample iteration calendar shows 2-week iterations, starting on Tuesday, accommodating various US and APAC holidays. Note that the cadence stays the same but the number of working days fluctuates based on the sample holidays. Customer company holidays, vacations, and sick time also influence the number of working days in a sprint / iteration.

Less than one week is possible if the work and coordination efforts can be optimized, more than 4 weeks is not recommended because longer sprints delay the feedback loop from working effectively and it is more like for engagement to "go dark" due to inactivity. Shorter sprints reinforce the sense of urgency and bias for action

2 week iterations are a good target duration to use unless the customer insists otherwise. For example, customers might want the GitLab engagement to follow the same iteration cadence as their own due to the integration into the customer's status and reporting process.

Managing a Globally Dispersed Team

What do you do if you just cannot make it work? Between various stakeholders, locations, time

zones, and other challenges, coordination can be difficult. Ultimately, the specific customer situation dictates what needs to be done. For example, working in different time zones might necessitate a flexible approach who works at what hours.

It is important to point out that certain meetings really, really, really need to be attended - iteration reviews for example demonstrate to the customer the tangible progress that has been achieved, so it is important that both GitLab and customer stakeholders attend in order to see that progress.

Part of the established cadence allows for frequent review and refinement of engagement priorities. Well understood, high priority, items bubble to the top of the priority list; and less defined items, with insufficient detail to be taken action on, drop to the bottom of the stack.

Review and refinement of the backlog allows for the dynamic reprioritization of items as the situation on the engagement adjusts.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/manage-risk/_index.md

Mitigating Risk via RAID Board

The RAID, Risks, Actions, Issues, and Decisions is our way to ensure a single source of truth for project risk & resolution

It is where our internal project stakeholders and leadership can reference the latest project information when the project is trending or sitting in a Y/R health status

The RAID is automatically created when the PM creates the CP template. First step is to rename template ·RAID - Customer - SOW/PO

While the RAID is created, managed, and reported by the PM, the internal & Customer team is encouraged to be updated the RAID as we work through Project challenges and mitigations

Example

Reporting on Risk via RAID

When creating a Risk, the following labels will be applied: "PS RAID Log" "PS Risk" "PSD Workflow::Not Started"

Severity should be defined and added as a label "PS:: Priority High" "PS:: Priority Medium" "PS:: Priority Low"

Impact will be quantified through impacts to Timeline, Scope, Quality, or Budget

An Owner will be assigned as a person responsible for accepting, mitigating, or resolving the Risk.

Mitigation plan will be detailed to lessen or eliminate the Risk.

Use the label ·Escalated· to create immediate attention as it impacts the progress of the project. For internal visibility, the RAID link is included in the Top Customer Report when a

project is Y/R

Escalated items can be found here.

Tracking Project Wins, Business Development Items, and Customer Stories

The process of a internal project retrospective is started when the project starts. It is a collection of team celebration, lessons learned, what could be improved, questions, comments, assets create. This in an internal-only issue and the GitLab team is encouraged to gather learnings and celebrations throughout the Project.

The Internal retro issue lives within the internal Customer EPIC and is created when the Epic is created, the latest template can be found at the issue level, and here
PMs to make sure this issue is linked in the internal Project slack channel

Reporting on Customer Wins, Lessons Learned, and Delivery Kit Updates

Customer Stories, Project Wins, and Completed Internal Retro report can be found here.
Needed Delivery Kit Updates can be found here.

For tracking lessons learned around scoping, PS process, PS Product, Support, or misaligned expectations, reference here.

SECTION: customer-success/professional-services-engineering/professional-services-delivery-methodology/retrospectives/_index.md

Simple Rules

1. The Program Manager / Project Manager acts as the meeting facilitator

1. The Retrospective scope if focused on the last sprint or the finalized engagement

1. Meeting participants include:

Program Manager / Project Manager

Technical Architect

Engagement Team

During the iteration level retrospectives, it is possible to have other participants, such as customers or other stakeholders, but that is entirely up to the team to decide

During final engagement level retrospectives, only GitLab or Partner team members should participate

1. During the Retrospective, the team focuses on the essential 3 questions:

What went well?

What would we like to change?

How can we implement that change?

1. The key to making a Sprint Retrospective useful is to keep it "action focused" (have a bias for action) · meaning the team needs to focus in on what they can change to make their own

process better

1. The meeting time is fixed: 45 minutes for each week of sprint duration

Managing Process Improvements over Time

As mentioned, the Retrospectives should be action focused - the intent is to rapidly improve upon the process in order to enable better productivity.

But, the reality is that some improvement suggestions might be quickly addressed, some might take some time, and others might be outside of the teams' area of decision power or exceed the engagement duration.

It is suggested to keep a running log of improvement suggestions that will help you keep track of what improvement ideas came up, what category they fall into, and what sprint / release they first came up in, etc.

This helps keep the team accountable and ensures you remember all the process improvements you implemented over time.

The time frames that some improvement suggestions fall into are as follows:

Within the Next Iteration / Sprint

An improvement suggestion that can be accomplished over the course of the next iteration, for example, refactoring of a specific CI job. Quick hits, quick successes.

Over Several Iterations / Sprints

Improvements that might take two or more iterations / sprints to implement. Re-architecting subsystems, or a comprehensive obtaining lengthy approvals, tasks that cannot be finished in one sprint, fall into this category.

Over the Next Engagement

Something that you coordinate beyond the current engagement, for example, improving the onboarding process, so you do not interrupt the delivery flow of your current engagement.

Never

Yes, some people do not like the idea that some improvement suggestions will never be addressed, but we all know situations like this. For example, if an automation task would be nice to implement but the implementation effort takes so long that we cannot realize a ROI, we'll probably never implement it.

This is an improvement suggestion that falls into the "Never" category; not because it might never happen, but because it is a decision in terms of impact and scope that cannot be decided upon by the team.

Common Challenges and How to Deal with Them

Boiling the Ocean

It is incumbent on the Program Manager / Project Manager to keep the team on track during Retrospectives. Sometimes teams have a tendency to focus in on problems that fall into the "Boiling the Ocean" category.

Those are sometimes great discussion topics (especially for a long night over a cup of mint tea), but they almost always fall outside of the scope and sphere of influence of the team.

In order to make sure that the Retrospective is useful and produces good improvement suggestions, the team should avoid these kinds of discussion topics and focus on two parameters to guide the discussion

1. Is the improvement suggestion relevant to the scope (last iteration / current engagement)?

1. Is the improvement suggestion achievable by the team?

Non-Participation

Several years ago a team member refused to participate in a Retrospective. When I asked him why, he said:

"Retrospectives are supposed to be like Lessons Learned meetings· but we never learn any lessons, we still got the same issues we had 2 years ago, so why waste time?"

His point was completely valid. To make Sprint Retrospectives work, you need to focus on what is relevant (last iteration / current engagement) and what your team can actually implement. Keeping track in a log as to what improvement suggestions were brought up, which ones got implemented, etc. keeps the team accountable but also shows the progress.

Blame Game

Improvement suggestions should never smell like blame is being assigned. It is important to have open team communication, and to honestly assess what can be improved upon by the team. Be careful not to assign blame but rather focus on potential solutions.

Everything is Great / Everything is Bad Attitude

Beware of both, irrational exuberance, and downtrodden doomsday forecasts. Both extremes are usually unrealistic. You can always improve on the process, and things are never as bad as some folks might make them seem.

No Improvement from Last Retrospective

Once again, track what you want to improve on, and if it is within the scope of one sprint, put it into the backlog for the next sprint.

As a general guidance, try to reserve up to 20% of capacity in early sprints for process improvements. In later sprints that might go down to 10%. Point here is that you have to account for this work. It is not going to happen by itself, or in addition to an already fully

utilized team.

Process improvement activities are part of the iteration activities, not in addition to it.

SECTION: customer-success/professional-services-engineering/professional-services-operations/_index.md

Welcome to GitLab Professional Services Operations

Professional Services Operations is a collaboration of our consulting, training, scheduling, reporting and backend processes to support the Professional Services team.

Who we are

Internal Page

Training Planning and Scheduling

Refer to this internal page for more information on Professional Services Operations' processes.

Operations

New Hire

When there is a new team member on the PS Team. We have some training and process links to review:

New Hire Review

Kantata Access

To provide Kantata access to an internal GitLab team members, provide access by the following:

- Kantata Access
 - Settings
 - Members
 - Invite Account Members
- Okta-Kantata- Users Google Group
 - Gmail
 - Gmail Apps
 - Groups

- Okta-Kantata- users
- Members
- Add Members

To provide Kantata access to a GitLab partner, provide access by the following:

- Kantata Access
 - Settings
 - Members
 - Invite Account Members
- Process a GitLab Access Request
 - Request Okta
 - Request Kantata to be added to Okta

Time Tracking

Accurate time tracking records of hours is essential to ensure revenue can be recognized based upon percentage completion of a project scope as outlined in a Statement of Work ("SOW"), and this data is used in the calculation of gross margin. Key points for time tracking include:

- Best practice is to record time at the end of each day. This provides the most accurate account of hours worked and when it was worked
- Each PSE is required and responsible for tracking their own hours, and submitted weekly by Friday EOD for the week worked
- If time will be worked over the weekend, time sheet should still be submitted by Friday, EOD, then a new line created on the time sheet for hours worked over the weekend
- Billable hours represent work hours that a staff member reports as being aligned to a specific SOW. The format for daily time tracking for each team member is shown below, and is reviewed weekly by PS Operations and Manager
- Hours for PTO, Holidays and Family and Friends day are no longer required to be submitted in the weekly time sheet
- If a team member did not work the time allocated for the week, then hours would be added to the PTO feature in Kantata
- Notes are not required for the PS Time Tracking - Non Creditable project
- Notes may be required for customer projects, please confirm with your Project/Program Manager before submitting time
- PTO should be submitted in the time off feature in Kantata, and also follow the company guidelines, time off process
 - Holidays along with Family and Friends day are scheduled on the Kantata calendar
- Time is required to round to the nearest quarter hour, example:
 - 15m should be .25
 - 30m should be .5
 - 45m should be .75

Kantata Internal Projects

Internal projects are set up to track internal time that is not customer project related. Below is the project link along with tasks and examples.

- PS Time Tracking - Non Creditable

Customer consulting projects

When working on a customer project, all hours worked should be tracked against the project. Here are some examples:

- Project tasks are aligned with SOW activities and hours tracked against the tasks
- Internal/ Sales Handoff Calls
- Internal/ External status meeting
- Support ticket submission while the project is in progress
- Weekly/ Final customer reports and documentation
- Status/ Close out customer calls
- Customer Travel

Customer training projects

When working on a training project, all hours worked should be tracked against the project. Here are some examples:

- Introduction/ Planning/ Preparation/ Close Out
 - All hours should be tracked against the task for preparation and close out of the training
 - All class hours are to be tracked against the task that gives the name of the training, example is GitLab Basics TRNG Hours

Project Expenses

Refer to this internal page for more information on onsite travel and expense processes.

If there are other questions in regards to the GitLab Travel policy, refer to the Travel Handbook Page.

Kantata Project Status/ Colors

Kantata Status Status Definition
----- -----
Estimate- Gray Projects that are tracking Internal time for GitLab PS & GitLab Partners
Prospect- Gray PC is setting up Kantata project/ Project is at a Stage 5- start reviewing staffing plan
In Set Up- Gray PC is setting up Kantata project/ Reviewing Staffing/ Welcome to PS Email
Okay to Start- Light Green Project setup complete/ PM Planning the project
Active- Dark Green PM/ PSE Actively working the project
Closed- Blue Project work is complete, waiting for billing and revenue to be complete
Completed- Blue Billing and Revenue is complete
On Hold- Gray Project is delayed
Backlog- Gray No work is planned
Cancelled- Blue Project Created but will not be worked for various reasons

For training-only projects, use the following Kantata/Kantata statuses:

Kantata/Kantata Status	Status Definition
Prospect- Gray	Project has not booked yet and is at a Stage 5
In Set Up- Gray	SFDC opportunity has closed / PC is setting up Kantata project
Okay to Start- Light Green	Project setup complete / Welcome to GitLab Education Services email
Active- Dark Green	PC is actively working on scheduling the project / Training dates are proposed to the customer
Closed- Blue	Onsite training projects only / Training delivery has been completed but waiting for expense reports
Completed- Blue	All training activities are completed / Ready for billing and revenue
On Hold- Gray	Partial training activities are completed; customer is not ready to schedule the remaining training activities
Backlog- Gray	Customer is not ready to schedule the training activities, and it has been at least a month since the order booking date
Cancelled- Blue	Project created but will not be worked for various reasons

Kantata Project Colors	
Blue	Training Only Project
Yellow	Consulting Only Project
Orange	Consulting & Training Project
Lime	Internal Project

Consulting Project Health Reports

Health Reports provide a weekly snap shot status to PS Management on the overall project status, and rolled up to executives. The report includes a Red, Yellow and Green indicator along with a section to update on overall status, schedule, scope, budget, client. Health reports should be filled out by the project lead or project manager by Thursday of week each by EOD for the projects that are in the Active project status.

- Overall Project Status:
 - Two to three lines describing the overall project status, includes pro/ cons/ blockers
- Project Schedule:
 - Is the project tracking to the current Kantata schedule? Y/N and if no, why?
- Project Scope:
 - Is the project tracking to the original scope, as per the SOW? Y/N and if no, why?
- Project Budget:
 - Is the project tracking to the original budget, as per the SOW? Y/N and if no, why?
- Client:
 - How is the customer feeling about the project? Happy, frustrated, engaged, disengaged
 - The project could be status of red for scope, schedule, etc. and the customer is still happy

New Supplier Request Form

- For new partners/vendors only
- Partner/vendor must be approved in Coupa prior to submitting a Professional Services Request and Purchase Request forms

- How to Request a New Supplier Instructions

New Professional Services Request Form

- Refer to the Coupa handbook page for instructions on using Coupa, GitLab's vendor invoicing sys
- This initiates the New Purchase Request form
- How to Create a Requisition
- How to complete the Professional Services Request Form
- Notes:
 - What is the individual being hires to do
 - GitLab Customer Name- Partner Name- PSE/ PM Name
 - Are any GitLab employees already performing this service?
 - For the most part, answer "no"
 - Will GitLab provide the individual with the required hardware/ device to complete the work?
 - For the most part, answer "no"
 - Is this a PS Partners service request?
 - For the most part, answer "no"
 - Is this an extension?
 - For the most part, answer "no"
 - Is the intended engagement meant to be over 6 months?
 - For the most part, answer "no"
 - Would you like Procurement to negotiate?
 - For the most part, answer "no"
 - What is the estimated spend?
 - For the most part, answer "no"
- Requisition Information
 - Add Partner Name
 - Description/ Item
 - GitLab -Partner Name- Customer Name- PSE/ PM Name
 - Training- Training: partner name / year month day
 - Price of the Partner SOW
 - Currency is always USD
 - Commodity would be COGS consulting fees
- Service Start Date:
 - Consulting: Estimate the customer project dates
 - Training: Estimate of when we will start using the training funds
- Service End Date:
 - Consulting: Estimate the customer project dates
 - Training: Estimate of when we will start using the training funds
- Need by- set this for 1 week out from date of your submission
- Payment Structure:
 - Data entry convention - Consulting: Hourly rate/ or Fixed Price
 - Example: \$/hour
 - Example: \$ Fixed Price
 - Data entry convention - Training:
 - Standard Course Delivery: Fixed Price
 - Pre-Configured Course Delivery: Hourly Rate
 - Custom Course Delivery: Hourly Rate
 - Example (standard course): \$ per course

- Example (pre-config or custom): \$/hour